

ANO
2026



UNINTER

**CADERNO DE RESPOSTAS DA
ATIVIDADE PRÁTICA DE:**

NoSQL

**ALUNO: VICTOR SPICHENKOFF SANTANA
4767042**

**Caderno de Resposta Elaborado por:
Prof. MSc. Guilherme Ditzel Patriota**



Atividade Prática – NEO4J

Questão 01 – IMPORTAÇÃO DOS ARQUIVOS JSON E CRIAÇÃO DE NÓS E ARESTAS COM BASE NOS DADOS DOS ARQUIVOS (Usar quantas páginas forem necessárias)

O QUE FAZER: Após configurar seu banco de dados em branco (novo DBMS em versão 4.*) e colocar no mínimo 3 e máximo 10 arquivos sequenciais do trabalho (dentre os 500 arquivos JSON disponibilizados) na pasta Import, crie um comando na linguagem do banco de dados Neo4j (Cypher), usando a biblioteca de importação de dados APOC, que leia os arquivos JSON desta pasta e crie os nós e relacionamentos com base nos dados neles. Faça a separação dos nós de mensagem em Tweet (mensagens originais), Retweet (mensagens repostadas), Quoted (mensagens que citam outras mensagens), Replied_to (mensagens de resposta à outras mensagens) com uso do campo `data[x].ref_tweet.type` (CUIDADO! Este campo só aparece no JSON de mensagens não originais e o uso de UNWIND para acessá-la pode impedir a criação dos nós de mensagens originais). Apenas com esta separação será possível resolver a questão 02 do trabalho.

Observação Importante: Seu banco de dados precisará conter todas as informações para resolver as questões 02 e 03. Leia elas antes, para entender o que você deseja fazer em cada uma e quais os nós, relacionamentos e atributos você irá importar dos arquivos JSON para conseguir resolver todo o trabalho sem a necessidade de recriar todo o seu banco de dados apenas para uma questão.

- I. Apresentação dos comandos (apenas query Cypher) usados (não esquecer do identificador pessoal/seu RU como parte do seu código, como um nome de atributo dos nós ou dado de um atributo de nós). **Nenhum dos comandos apresentados aqui pode conter a palavra RETURN (Peso na nota: 12.5%):**

```
/* 1 - CRIAR TWEET E USERS (genérico) */
CALL apoc.load.directory("*.json") YIELD value AS one_file
CALL apoc.load.json(one_file) YIELD value
UNWIND value.data AS tweet

MERGE (t:Tweet {tweet_id: tweet.id})
ON CREATE SET
    t.user_id = tweet.author_id,
    t.conversation_id = tweet.conversation_id,
    t.created_at = datetime(tweet.created_at),
    t.type = "original"

MERGE (u: User {user_id: tweet.author_id})

MERGE (u)-[:TWEETED]->(t);
```

Figura 1: Comando para criação de usuário e tweets, bem como da relação "TWEETED". Tweets genéricos, todos recebem type="original", caso esse campo não seja sobrescrito, considera-se um tweet original.

```
/* 2 - TIPAR TWEETS */
CALL apoc.load.directory("*.json") YIELD value AS one_file
CALL apoc.load.json(one_file) YIELD value
UNWIND value.data AS tweet
MATCH (t:Tweet {tweet_id: tweet.id})
FOREACH (ref IN coalesce(tweet.referenced_tweets, [])) |
    SET t.type = ref.type
);
```

Figura 2: Comando para adicionar tipagem correta aos tweets (apenas campo).



```
/* 3 - CRIAR HASHTAGS */  
CALL apoc.load.directory("*.json") YIELD value AS one_file  
CALL apoc.load.json(one_file) YIELD value  
UNWIND value.data AS tweet  
  
MATCH (t:Tweet {tweet_id: tweet.id})  
  
WITH t, coalesce(tweet.entities.hashtags, []) AS hashtags  
UNWIND hashtags AS hashtag  
WITH t,  
    toLower(  
        apoc.text.replace(  
            apoc.text.clean(hashtag.tag),  
            '^[a-zA-Z0-9]',  
            ''  
        )  
    ) AS cleanedHashtag  
WHERE cleanedHashtag <> ""  
MERGE (h:Hashtag {tag: cleanedHashtag})  
MERGE (t)-[:HAS_HASHTAG]->(h);
```

Figura 3: Comando para adicionar as hashtags e criar o relacionamento entre elas e os tweets ("HAS_HASHTAG").

```
/* 4 - Atualizar tipos para Retweet, Quoted e Replied_To*/  
MATCH (t: Tweet)  
WHERE t.type = "retweeted"  
REMOVE t: Tweet  
SET t: Retweet;  
  
MATCH (t: Tweet)  
WHERE t.type = "quoted"  
REMOVE t: Tweet  
SET t: Quoted;  
  
MATCH (t: Tweet)  
WHERE t.type = "replied_to"  
REMOVE t: Tweet  
SET t: Replied To;
```

Figura 4: Comando para realizar a separação entre Tweet, Retweet, Quoted e Replied_To.

```
/* 5 - Adição do meu RU */  
MERGE (eu4767042:RU {ru: 4767042});
```

Figura 5: comando para adicionar meu RU como um nó.

- II. Apresentação dos prints do resultado (não esquecer do identificador, seu RU). Estes prints devem ser de cada tela do Neo4j Browser após a execução bem-sucedida de cada comando (não mostrar nenhum grafo nesta parte, apenas as telas de execução dos comandos da parte I. O uso de RETURN zerará a nota desta parte) **(Peso na nota: 12.5%)**:

```
neo4j$ CALL apoc.load.directory("*.json") YIELD value AS one_file CALL apoc.load.json(one_file) YIELD value UNWIND value.data AS ...  
Added 772 labels, created 772 nodes, set 2352 properties, created 395 relationships, completed after 1525 ms.
```

Figura 6: saída comando de criação de usuário e tweets.

```
neo4j$ CALL apoc.load.directory("*.json") YIELD value AS one_file CALL apoc.load.json(one_file) YIELD value UNWIND value.data AS ...  
Set 371 properties, completed after 371 ms.
```

Figura 7: saída alteração do campo type em tweets.

```
neo4j$ /* 3 - CRIAR HASHTAGS */ CALL apoc.load.directory("*.json") YIELD value AS one_file CALL apoc.load.json(one_file) YIELD va...  
Added 93 labels, created 93 nodes, set 93 properties, created 516 relationships, completed after 471 ms.
```

Figura 8: Saída criação de hashtags e relacionamentos.

```
$ MATCH (t: Tweet) WHERE t.type = "retweeted" REMOVE t: Tweet SET t: Retweet; MATCH (t: Tweet) WHERE t.type = "quoted" REMOVE t: ...  
neo4j$ MATCH (t: Tweet) WHERE t.type = "retweeted" REMOVE t: Tweet SET t: Retweet  
SUCCESS  
Added 331 labels, removed 331 labels, completed after 22 ms.  
neo4j$ MATCH (t: Tweet) WHERE t.type = "quoted" REMOVE t: Tweet SET t: Quoted  
neo4j$ MATCH (t: Tweet) WHERE t.type = "replied_to" REMOVE t: Tweet SET t: Replied_To
```

Figura 9: Saída separação dos tweets.

```
neo4j$ MERGE (eu4767042:RU {ru: 4767042});  
Added 1 label, created 1 node, set 1 property, completed after 6 ms.
```

Figura 10: Saída da criação de RU.



Node Labels

*(866) Hashtag Quoted RU
Replied_To Retweet Tweet
User

Relationship Types

*(911) HAS_HASHTAG
TWEETED

Property Keys

conversation_id created_at ru
tag tweet_count tweet_id
type user_id

Figura 11: Informações do banco no Neo4J após executar todos os comandos.

Atividade Prática – NEO4J

Questão 02 – DESCOBERTA DA HASHTAG PRINCIPAL (Usar quantas páginas forem necessárias)

ENUNCIADO: Você deve criar e executar um comando Cypher em seu banco de dados para descobrir qual hashtag está presente em todas as mensagens originais (excluindo mensagens de retweet, citação e resposta). Este comando **não deve** fazer uso da biblioteca **APOC**. Caso seu comando tente retornar mais de 300 nós, a configuração padrão do Neo4j Browser impedirá a exibição de mais de 300 nós. Seu comando **não pode** ser **MATCH (n) RETURN n;**.

- I. Apresentação do comando (apenas query Cypher) usado (não esquecer do identificador pessoal no comando, seu RU). **Este comando deve conter ao menos uma dupla MATCH/RETURN (sugerimos uso de subquery para resolução completa) (Peso na nota: 12.5%):**

```
MATCH (t:Tweet {type:'original'})-[:HAS_HASHTAG]->(h:Hashtag)
WITH h, count(*) AS total
ORDER BY total DESC
LIMIT 1

MATCH (t:Tweet {type:'original'})-[r:HAS_HASHTAG]->(h)
WITH t, h, r
LIMIT 12
MATCH (eu4767042: RU)

RETURN h, t, r, eu4767042;
```

Figura 12: Na primeira parte, realiza-se uma contagem das hashtags usadas, considerando apenas tweets originais. O resultado limita-se a 1. Na segunda parte, busca-se tweets associadas a aquela hashtag. O terceiro match refere-se apenas a consulta do RU.

- II. Apresentação do grafo gerado contendo apenas 1 nó de hashtag ao centro (a sua resposta) e ao menos mais 10 nós de mensagens relacionadas a este nó de hashtag. O print deve conter o grafo sem zoom e a legenda de tipos de nós e cores geradas pelo neo4j, incluindo a informação da quantidade de nós de Hashtag mostrados (não esquecer do identificador pessoal, seu RU) **(Peso na nota: 12.5%):**

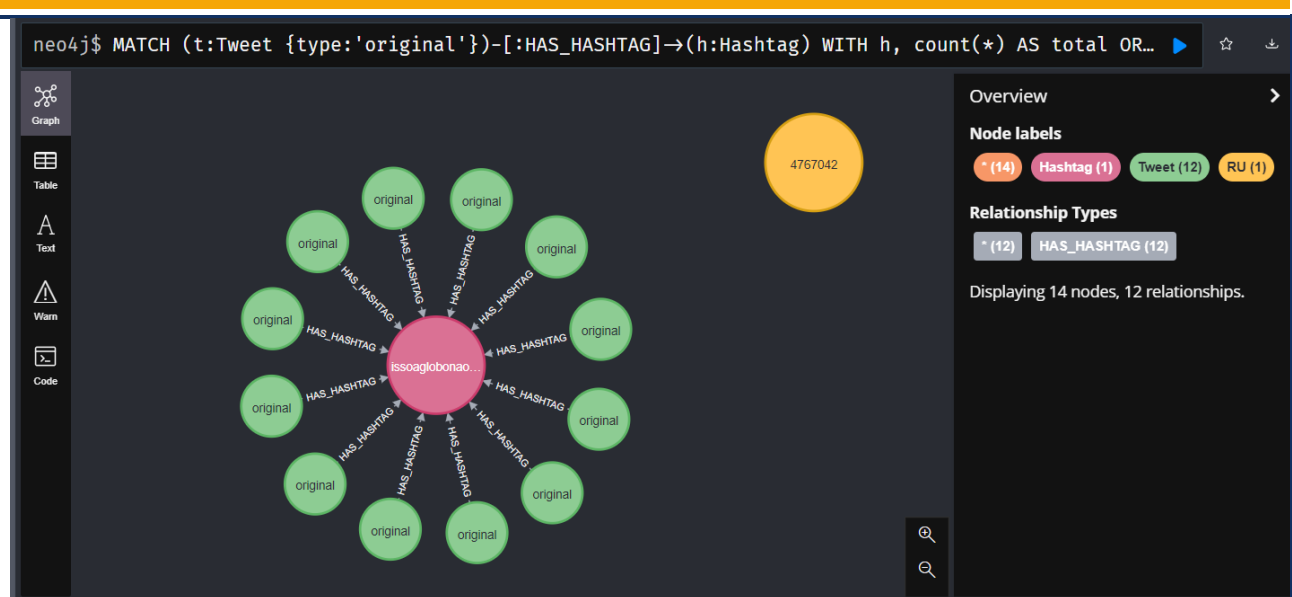


Figura 13: Grafo resultante. Indica 12 tweets ligados a essa hashtag (devido ao limit), mostra a hashtag principal e também RU.

III. Responda à pergunta: Qual foi a hashtag usada como filtro para coleta dos dados analisados? (Esta hashtag só estará presente em nós do tipo Tweet e não em Retweet. Sua resposta deve conter apenas 1 palavra com o texto da hashtag) (Peso na nota: 12.5%)?

Resposta (apenas 1 palavra): issoaglobonaomuestra

Atividade Prática – NEO4J

Questão 03 – ANÁLISE DOS DADOS SEGUNDO VIÉS A SUA ESCOLHA (Usar quantas páginas forem necessárias)

ENUNCIADO: Usando o mesmo banco de dados já criado na questão 01 e usado na questão 02, busque alguma informação que você julgue relevante nos dados (seu comando **não pode** ser **MATCH (n) RETURN n**; nem pode conter a biblioteca **APOC**). Sua tarefa aqui é analisar os dados do banco e definir qual informação você gostaria de obter e que tenha potencial de gerar um grafo com 10 ou mais nós interligados entre si. Sua análise deve responder à uma pergunta clara, como por exemplo:

- Qual o dispositivo mais usado para tuitar? (depende de seu banco de dados já possuir os nós de equipamentos usados, criados na questão 01).
 - Qual a Hashtag que menos foi usada?
- Qual o usuário mais movimentou a rede? (depende de você ter criado nós de usuário na questão 01)
 - Quais os usuários mais citados? (depende dos seus nós de mensagem possuírem esta informação ou de relacionamentos terem sido criados para este fim).

- I. Apresentação dos comandos (apenas queries Cypher) usados por você para realizar sua análise (não esquecer do identificador pessoal, seu RU) (**Peso na nota: 12.5%**):

```
/* Qual a hashtag mais usada considerando todas as mensagens? */  
MATCH (t)-[:HAS_HASHTAG]->(h:Hashtag)  
WITH h, count(*) AS total  
ORDER BY total DESC  
LIMIT 1  
  
MATCH (t)-[r:HAS_HASHTAG]->(h)  
WITH t, h, r, total  
LIMIT 10  
MATCH (eu4767042: RU)  
  
RETURN h, t, r, eu4767042, total;
```

Figura 14: Query para buscar a hashtag mais usada considerando todas as mensagens.

- II. Apresentação do print do resultado, podendo ser uma tabela ou um grafo. O print deve conter o resultado e o comando executado. (não esquecer do identificador, seu RU) **(Peso na nota: 12.5%)**:

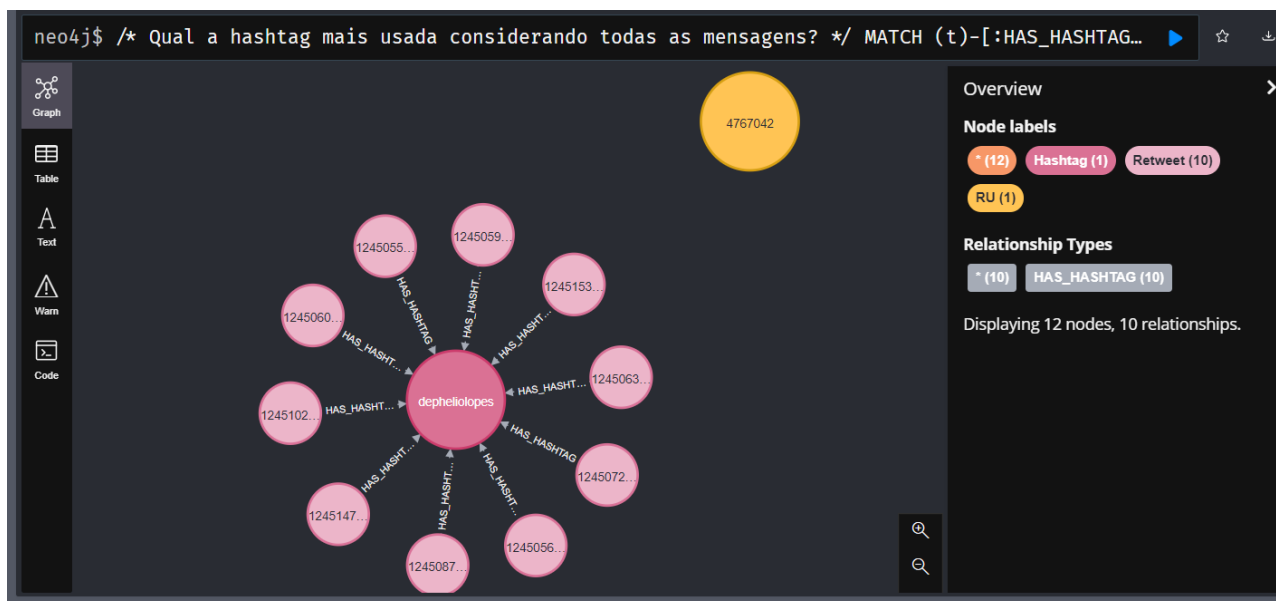


Figura 15: Resultado. Demonstra que a hashtag mais usada é “depheliolopes”.

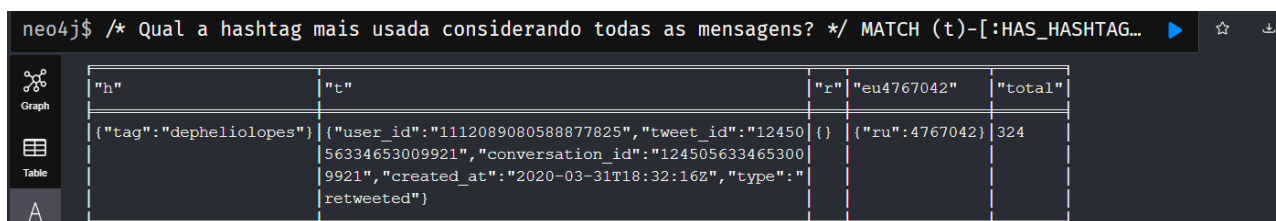


Figura 16: Tela alternativa, demonstra que a hashtag foi usada 324 vezes.

- III. Explique qual foi a análise realizada, incluindo sua linha de raciocínio para criação da query Cypher e qual era sua expectativa de resultado antes da análise, comparando-a com o resultado realmente obtido após execução do comando **(Peso na nota: 12.5%)**.

Resposta: Buscou-se analisar qual seria a hashtag mais usada considerando todas as mensagens. Para tanto, foi importante realizar a consulta sem filtrar o tipo de mensagem. O funcionamento da query é simples, realizando uma consulta para descobrir a hashtag mais usada e depois coletar alguns tweets associados para demonstração. Esperava-se que a hashtag principal dos tweets fosse também a mais usada, porém, descobriu-se que não. Contudo, a hashtag “depheliolopes” (deputado Helio Lopes) tem ligeira conexão com a hashtag “issoaglobonaomostro” e seu contexto. Outro detalhe interessante deve-se ao fato de quase todos os tweets associados a essa hashtag serem do tipo Retweet, ao menos na amostragem coletada.